

Unified Identity, Authorization and Visibility Architecture

1. Purpose

The Debt Collections Platform consists of multiple applications such as CRM, SEAM, DCM, Field Collections, Litigation, Dialer integrations, and supporting operational applications.

As the platform grows, access control cannot remain application-specific. We need a common platform capability that consistently answers three fundamental questions:

Capability	Question
Identity	Who is the user?
Authorization	What is the user allowed to do?
Visibility	Which cases is the user allowed to access?

The objective of this architecture is to provide a **simple, centralized and configurable access-control model** that can work across different customers without forcing every customer into the same organizational hierarchy.

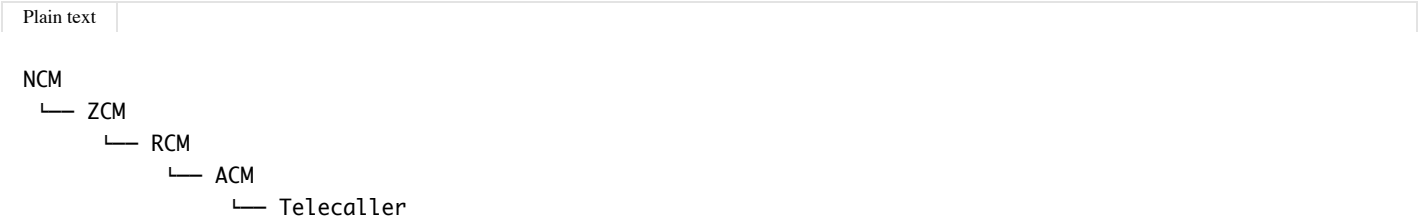
2. Problem Statement

Different customers organize their collections teams differently.

Examples:

- A Telecaller may report to one immediate manager.
- The same Telecaller may be visible to multiple managers.
- An Area Collections Manager may need visibility across all cases belonging to a branch.
- Visibility may depend on Product, Bucket, Agency, Branch, Assignee, or combinations of these.
- A Tenant Administrator may need visibility across the entire tenant.

Therefore, a traditional fixed hierarchy such as:



cannot be the foundation of access control.

The platform instead needs a **configurable visibility model based on case attributes**.

3. Architecture Principles

The architecture follows five key principles.

3.1 Identity, Authorization and Visibility are separate concerns

Authentication should not determine case visibility.

RBAC should not determine organizational hierarchy.

Visibility should not determine whether the user is allowed to perform an operation.

Each capability has one clear responsibility.

3.2 Visibility is based on case scope, not hierarchy

The system does not need to understand whether a user is an ACM, RCM or Manager to determine visibility.

Instead, the configuration explicitly defines:

Which cases can this user see?

For example:

Plain text

M1 can see Branch B1

or:

Plain text

M2 can see cases assigned to T1, T2 and T3

or:

Plain text

M3 can see Personal Loan cases in Bucket B1 and B2 for Branch B4

3.3 Customer-specific organization structures remain configurable

The platform should not impose one organizational model across tenants.

Customer A may use immediate-manager visibility.

Customer B may have two managers responsible for the same team.

Customer C may allocate visibility based on Branch.

All should work using the same visibility engine.

3.4 Visibility must always be enforced by backend services

Frontend visibility is only for user experience.

The backend remains authoritative.

A user manipulating frontend parameters must never be able to access cases outside their configured visibility.

3.5 Configuration must remain business-friendly

Customers should not have to understand:

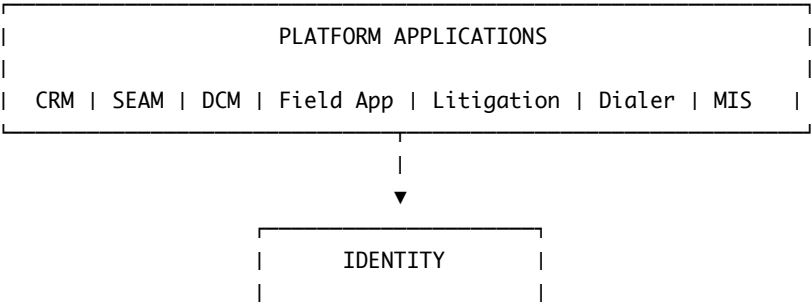
- policies
- scopes
- relationship graphs
- hierarchy engines
- authorization expressions

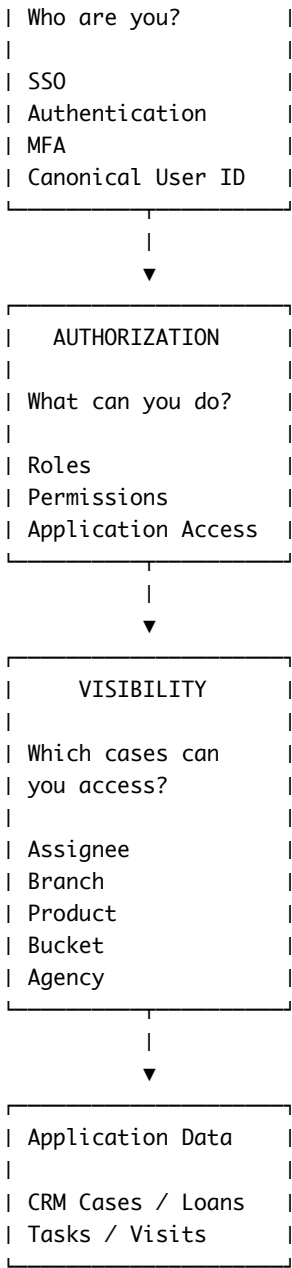
They should simply configure:

User X can see cases matching Y.

4. Target Architecture

Plain text





5. Identity Layer

Responsibility

The Identity Layer answers:

Who is the user?

All platform applications should trust one common identity provider or one logical identity platform.

A user should have one canonical platform identity.

Example:

Plain text	
userId	: USER-10231
tenantId	: CUSTOMER-A
username	: user@customer.com
status	: ACTIVE

Applications may maintain their own user profile data, but security decisions should reference the canonical platform user ID.

Key capabilities

The Identity Layer is responsible for:

- Login
- SSO
- MFA where required
- Session/token issuance
- User lifecycle
- Tenant association
- Canonical user identification

It is **not responsible for deciding which cases the user can see.**

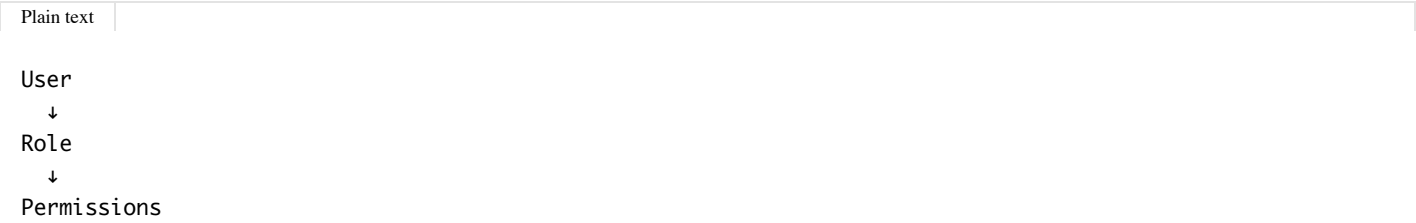
6. Authorization Layer

Responsibility

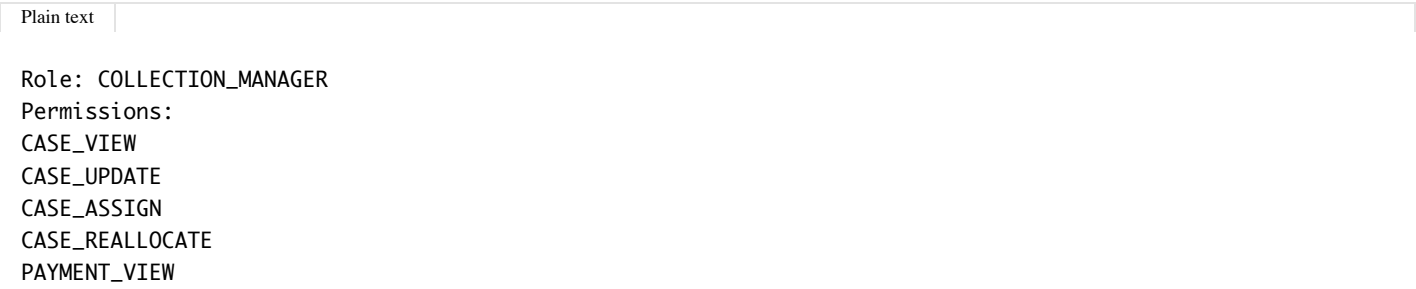
Authorization answers:

What actions can the user perform?

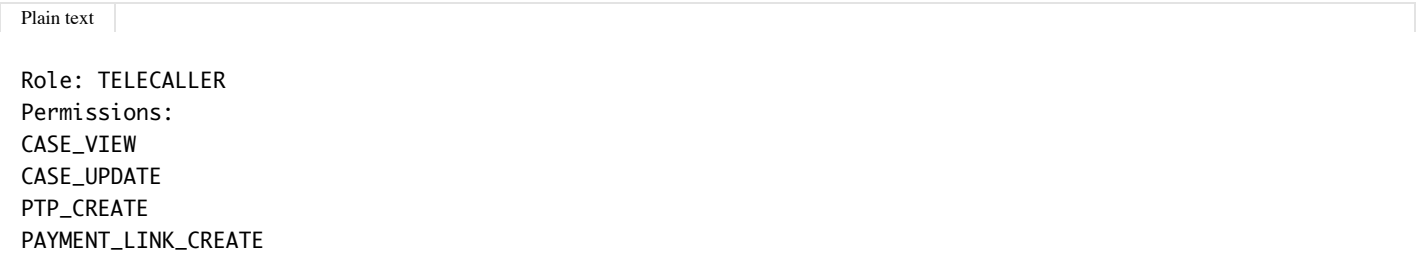
The proposed model is centralized RBAC.



Example:



Example Telecaller:



Authorization determines whether the operation is allowed.

It does not determine the data scope.

For example:



does not mean:



M1 can see every case in the tenant.

Visibility determines that separately.

7. Visibility Layer

Responsibility

Visibility answers:

Which cases can this user access?

The Visibility Layer evaluates configured case dimensions.

Initial supported dimensions are:

Plain text	
ASSIGNEE	
BRANCH	
PRODUCT	
BUCKET	
AGENCY	

Additional dimensions can be introduced later without redesigning the overall architecture.

8. Visibility Rule Model

A visibility rule is conceptually:

Plain text	
USER + CASE CONDITIONS	

Example:

Plain text	
M1	
Branch = B1	

Meaning:

M1 can access all cases tagged to Branch B1.

Another example:

Plain text	
M2	
Assigned Users = T1, T2, T3	

Meaning:

M2 can access cases assigned to T1, T2 or T3.

Another:

Plain text	
M3	
Branch = B1	
Product = PL	
Bucket = B1, B2	

Meaning:

M3 can access Personal Loan cases belonging to Branch B1 and Bucket B1 or B2.

9. AND / OR Rule Semantics

The visibility syntax must remain predictable.

The final rule is:

- Values within a column are OR.
- Different columns within one rule are AND.
- Multiple rules for the same user are OR.

9.1 Same dimension = OR

Example:

Plain text

```
Branch = B1 | B2
```

means:

Plain text

```
Branch = B1
OR
Branch = B2
```

Equivalent:

Plain text

```
branch_id IN ('B1', 'B2')
```

9.2 Different dimensions = AND

Example:

Plain text

```
Branch = B1
Product = PL
Bucket = B1 | B2
```

means:

Plain text

```
Branch = B1
AND Product = PL
AND (Bucket = B1 OR Bucket = B2)
```

Equivalent:

Plain text

```
branch_id = 'B1'
AND product = 'PL'
AND bucket IN ('B1', 'B2')
```

9.3 Multiple rules = OR

Example:

User	Branch	Product	Bucket
M1	B1		

M1 B2 PL B3

means:

Plain text

Branch = B1
OR
(
 Branch = B2
 AND Product = PL
 AND Bucket = B3
)

This gives M1:

- all B1 cases
- plus PL/B3 cases from B2

10. Common Visibility Scenarios

Scenario 1 — Telecaller

T1 should see only cases assigned directly to T1.

Default rule:

Plain text

ASSIGNEE = SELF

Effective condition:

Plain text

assigned_to = :loggedInUser

This default can be applied automatically for Telecaller and Field Agent roles where appropriate.

Scenario 2 — Immediate Manager

M1 manages T1, T2 and T3.

Configuration:

Plain text

User: M1
Assigned Users:
T1 | T2 | T3

Effective condition:

Plain text

assigned_to IN ('T1', 'T2', 'T3')

No formal hierarchy model is required.

Scenario 3 — Multiple Managers

Both M1 and M2 should see work assigned to T1, T2 and T3.

Configuration:

Plain text

M1 → T1 | T2 | T3
M2 → T1 | T2 | T3

Both managers receive equivalent visibility.

Scenario 4 — Branch Visibility

M1 can see all cases tagged to Branch B1.

Plain text

M1
Branch = B1

M1 and M2 can both see Branch B2.

Plain text

M1
Branch = B2
M2
Branch = B2

No relationship between M1 and M2 is required.

Scenario 5 — Product + Bucket + Branch

M3 should see Personal Loan cases for Branch B1 in Bucket B1 and B2.

Plain text

User = M3
Branch = B1
Product = PL
Bucket = B1 | B2

Effective visibility:

Plain text

branch_id = 'B1'
AND product = 'PL'
AND bucket IN ('B1','B2')

Scenario 6 — Tenant Administrator

Tenant Admin requires access to all cases belonging to the tenant.

Instead of interpreting an empty configuration as unrestricted access, use an explicit rule:

Plain text

ALL_CASES = TRUE

Effective condition:

Plain text

tenant_id = :loggedInTenant

This avoids accidental unrestricted access caused by incomplete configuration.

11. Bulk Visibility Configuration

Customers may need to configure visibility for hundreds or thousands of users.

The customer-facing configuration should therefore remain simple.

Recommended Excel structure:

User ID	Branch	Product	Bucket	Agency	Assigned Users	Self Assigned
M1	B1					
M1	B2	PL	B3			
M2	B2					
M3					T1 T2 T3	
ADMIN1						
TC1						Y

The business does not need to create:

- visibility groups
- policies
- hierarchy structures
- rule identifiers

Each row simply means:

This user can see cases matching these conditions.

12. Upload Processing

Visibility configuration should not be applied directly from Excel to runtime tables.

Recommended flow:



The full file should be validated before publishing.

Example response:

Plain text

```
Rows processed : 520
Valid rows      : 517
Invalid rows    : 3
Row 27  : User M99 does not exist
Row 64  : Branch B78 does not exist
Row 216 : Product XX is invalid
```

Nothing should go live until validation succeeds.

13. Versioning

Each published configuration should create a version.

Example:

Plain text

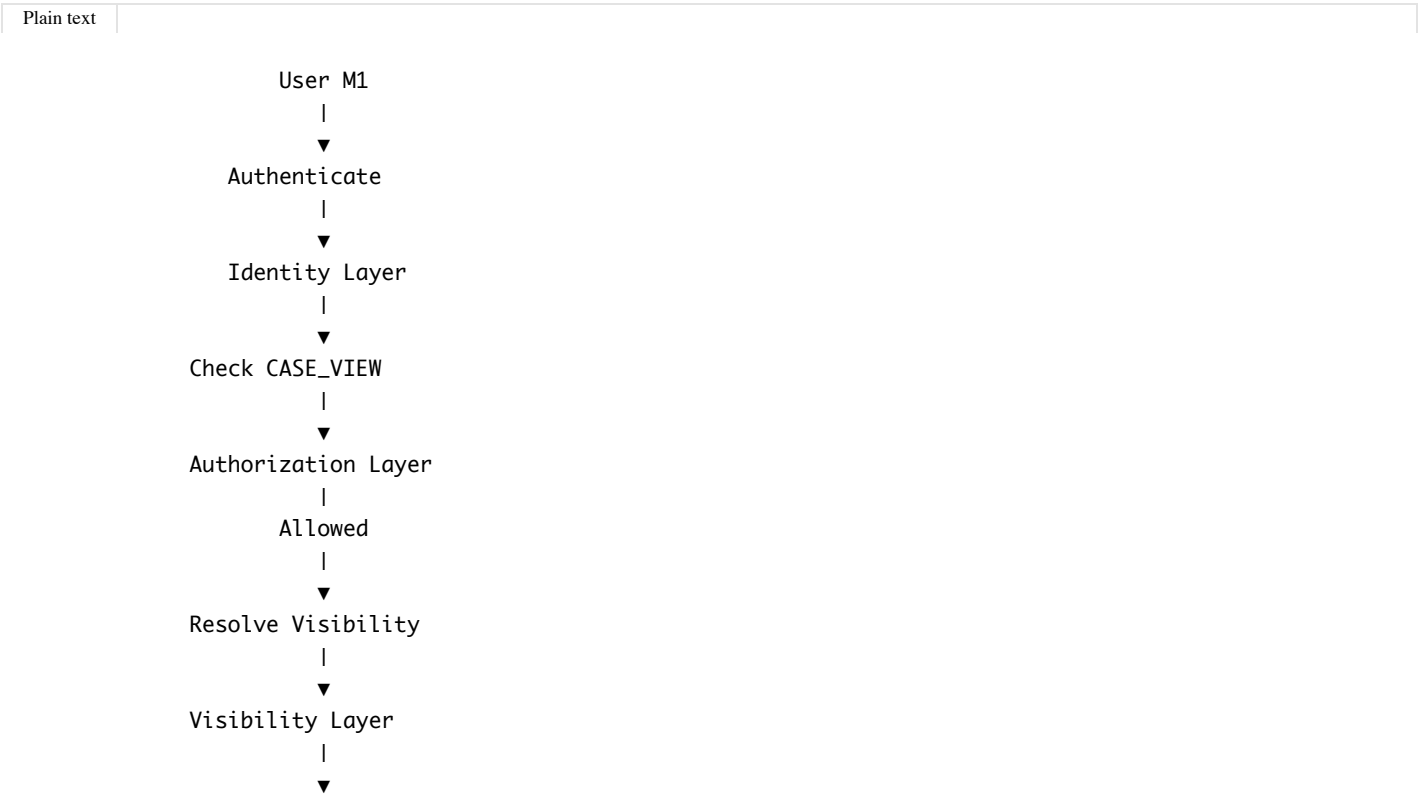
```
Visibility Configuration
Version 21
Status: PREVIOUS
Version 22
Status: ACTIVE
```

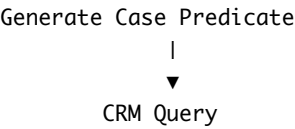
Benefits:

- complete audit trail
- traceability
- rollback capability
- controlled deployment
- easier production troubleshooting

14. Runtime Flow

Example: M1 opens the CRM All Cases page.





Suppose M1 has:

Plain text

Rule 1:
Branch = B1
Rule 2:
Branch = B2
Product = PL

The Visibility Layer resolves:

Plain text

Branch B1
OR
(
 Branch B2
 AND Product PL
)

CRM applies:

Plain text

WHERE tenant_id = :tenantId
AND
(
 branch_id = 'B1'
 OR
 (
 branch_id = 'B2'
 AND product = 'PL'
)
)

15. Runtime Integration Pattern

Applications should not perform a visibility check once per case.

Avoid:

Plain text

CASE1 → Visibility API
CASE2 → Visibility API
CASE3 → Visibility API
...

For large case lists this would perform poorly.

Instead:

Plain text

User
↓
Visibility Resolver
↓
Effective Visibility Context
↓
Database Predicate

↓
Query Cases

Visibility should be resolved once and converted into an efficient database/search predicate.

16. Visibility Context

The platform may expose an internal representation such as:

Plain text

```
{
  "userId": "M1",
  "tenantId": "CUSTOMER_A",
  "rules": [
    {
      "branch": ["B1"]
    },
    {
      "branch": ["B2"],
      "product": ["PL"],
      "bucket": ["B1", "B2"]
    }
  ]
}
```

This representation is internal.

Customers continue to use the simple business-facing Excel.

17. Proposed Logical Data Model

A minimal model could contain:

visibility_rule

Plain text

```
rule_id
tenant_id
user_id
all_cases
version
active
```

visibility_rule_condition

Plain text

```
rule_id
dimension
value
```

Example:

Plain text

```
RULE-101 | BRANCH | B1
RULE-101 | PRODUCT | PL
RULE-101 | BUCKET | B1
RULE-101 | BUCKET | B2
```

This represents:

Plain text

```
Branch = B1
AND Product = PL
AND Bucket IN (B1,B2)
```

A second rule for the same user automatically introduces OR semantics.

18. Caching

Visibility configuration is read frequently but changes relatively infrequently.

Therefore resolved visibility can be cached.

Example:

```
Plain text

Cache Key:
tenantId:userId:visibilityVersion
```

For example:

```
Plain text

CUSTOMER_A:M1:22
```

When Version 23 becomes active:

```
Plain text

Version 22 → previous
Version 23 → active
```

the existing cached context becomes invalid automatically.

This avoids repeated evaluation of configuration on every request.

19. Security Guardrails

Several safeguards should be mandatory.

Tenant isolation

Every visibility query must always include:

```
Plain text

tenant_id = loggedInTenant
```

Visibility configuration must never allow cross-tenant access.

Explicit global visibility

Blank configuration must never imply:

```
Plain text

ALL CASES
```

Full visibility must require:

```
Plain text

ALL_CASES = TRUE
```

Backend enforcement

Frontend controls must never be considered security enforcement.

Every backend query must apply the effective visibility context.

Fail closed

If visibility configuration cannot be resolved:

Plain text

DENY

rather than returning unrestricted data.

Auditability

Changes should record:

Plain text

Who changed visibility?
When?
Which tenant?
Previous version?
New version?
Upload source?

20. Responsibility Matrix

Capability	Identity Authorization Visibility		
Authenticate user	Yes	No	No
SSO	Yes	No	No
Maintain roles	No	Yes	No
Maintain permissions	No	Yes	No
Determine CASE_VIEW permission	No	Yes	No
Determine visible cases	No	No	Yes
Branch-based access	No	No	Yes
Product-based access	No	No	Yes
Bucket-based access	No	No	Yes
Agency-based access	No	No	Yes
Assignee-based access	No	No	Yes
Tenant-wide case visibility	No	No	Yes

21. What We Are Deliberately Not Building

At this stage the solution does **not** introduce:

- Organizational hierarchy engine
- Relationship graph
- Generic policy engine

- ABAC framework
- ReBAC framework
- Command Tower integration
- Filter configuration framework
- Assignment engine

These may become future capabilities if justified by new requirements.

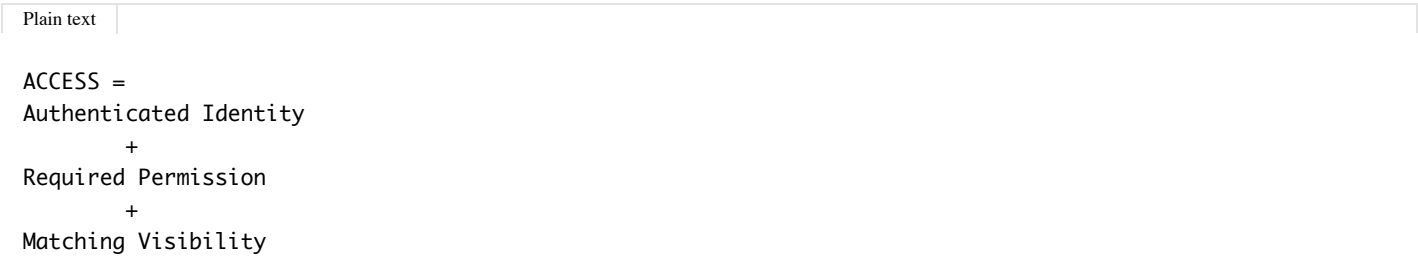
For the current requirement, introducing them would increase complexity without adding sufficient value.

22. Final Architecture Position

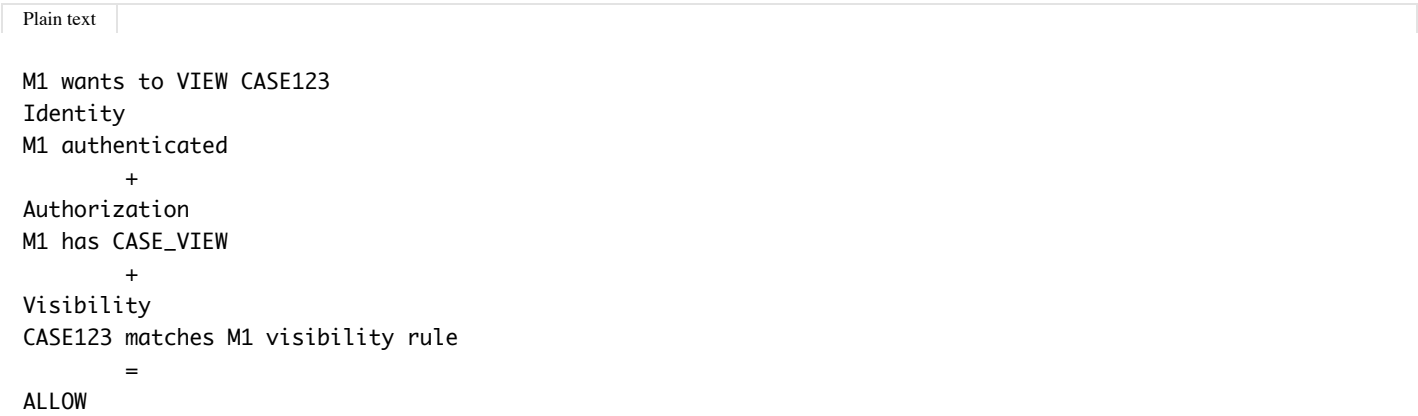
The platform access-control model is therefore intentionally simple:



Or expressed as a single runtime decision:



Example:



This design gives us a centralized platform capability while still allowing every tenant to define its own operating model.

It remains easy for Product teams to explain, simple for customers to configure, and predictable for Engineering teams to implement and operate.

Recommended Next Step

The next level of design should focus only on the **Visibility Service implementation**, specifically:

1. APIs

2. PostgreSQL data model
3. Excel upload contract
4. Visibility resolution algorithm
5. Caching strategy
6. How CRM converts visibility rules into efficient queries

That would turn this architecture into an implementation-ready design without changing the three-layer model above.